

Lab10_Webclient(Sec 1) - Classr x LAB10_WebClient/README.md x

localhost:8080/products

Pretty-print ☒

```
[
  {
    "id": "1",
    "name": "iPhone 15 - Rattaphum 673380057-5 SEC 1",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

Lab10_Webclient(Sec 1) - Classr x LAB10_WebClient/README.md x

localhost:8080/products/1

Pretty-print ☒

```
{
  "id": "1",
  "name": "iPhone 15 - Rattaphum 673380057-5 SEC 1",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900,
  "discountType": "MEMBER",
  "discountedPrice": 35910
}
```

Lab10_Webclient(Sec 1) - Classr x LAB10_WebClient/README.md x

localhost:8080/products/category/Electronics

Pretty-print ☒

```
[
  {
    "id": "1",
    "name": "iPhone 15 - Rattaphum 673380057-5 SEC 1",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

Lab10_Webclient(Sec 1) - Classr x LAB10_WebClient/README.md x

localhost:8080/products/1/price

Pretty-print ☐

35910.0

Home Workspaces API Network Search Postman Ctrl K

Rattaphum Kerdprajeen's Workspa... New Import

Overview My Collection POST New Request GET New Request

Collections + Search collections

My Collection

- GET Get data
- POST Post data
- GET New Request
- GET New Request
- GET New Request
- GET New Request

Environments

History

Flows

My Collection / New Request

POST http://localhost:8080/products

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "id": "4",
3   "name": "iPad Pro",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 15,
7   "price": 35900.0,
8   "discountType": "MEMBER"
9 }
```

Body Cookies Headers (2) Test Results

{ } JSON Preview Visualize

```
1 {
2   "id": "4",
3   "name": "iPad Pro",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 15,
7   "price": 35900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 32310.0
10 }
```

Lab10_Webclient(Sec 1) - Class LAB10_WebClient/README.md Lab 10 Spring WebReact localhost:8080/products

localhost:8080/products

Pretty-print

```
[
  {
    "id": "1",
    "name": "iPhone 15 - Rattaphum 673380057-5 SEC 1",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  },
  {
    "id": "4",
    "name": "iPad Pro",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 15,
    "price": 35900,
    "discountType": "MEMBER",
    "discountedPrice": 32310
  }
]
```

API Network

Search Postman Ctrl K

Rattaphum Kerdprajeen's Workspa... New Import

Overview My Collection POST New Request GET New Request

My Collection / New Request

DELETE http://localhost:8080/products/4

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (1) Test Results

Raw Preview Visualize

1

Lab10_Webclient(Sec 1) - Classr LAB10_WebClient/README.md คู่มือทำ Lab 10 Spring WebReact

localhost:8080/products

Pretty-print

```
[
  {
    "id": "1",
    "name": "iPhone 15 - Rattaphum 673380057-5 SEC 1",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

1. Reactive Programming vs Blocking I/O ต่างกันอย่างไร?

- Blocking I/O (Traditional):
 - ใช้โมเดล 1 Thread per Request
 -
 - เมื่อมีการร้องขอ I/O (เช่น ค้นหา Database หรือเรียก HTTP API) Thread จะหยุดรอ (Blocked) จนกว่าจะได้รับข้อมูลกลับมา
 - ทำให้สิ้นเปลืองทรัพยากรสูงเมื่อเจอระบบที่มี High-Concurrency
- Reactive Programming (Non-blocking):
 - ใช้โมเดล Event-driven / Asynchronous Stream ที่ไม่บล็อก Thread
 - Thread จะไม่จอดรอผลลัพธ์ แต่จะสลับไปทำงานอื่นได้ทันที
 - เมื่อข้อมูลพร้อม ค่อยส่ง Signal (onNext, onComplete) กลับมาแจ้ง Subscriber
 - รองรับจำนวน Request พร้อมกันได้มหาศาลโดยใช้จำนวน Thread เพียงน้อยนิด

2. Mono vs Flux ใช้กรณีไหน และทำไม?

- Mono:
 - ใช้เมื่อ: ต้องการคืนค่าผลลัพธ์เพียง 0 หรือ 1 ค่า
 -
 - เหตุผล: เหมาะกับงานจำพวก findById, save, update, delete หรือการคำนวณที่ส่งผลลัพธ์เดียว
- Flux:
 - ใช้เมื่อ: ต้องการคืนค่าผลลัพธ์เป็นข้อมูลต่อเนื่อง 0 ถึง N ค่า (Data Stream)
 - เหตุผล: เหมาะกับงานคิวรีรายการคอลเลกชันอย่าง findAll, findByCategory หรือข้อมูลสตรีมมิงแบบ Real-time

src > main > java > com > example > lab10 > repository > J ProductRepository.java > Language Support for Java(TM) by Red Hat > ProductRepository

```
1 package com.example.lab10.repository;
2
3 import java.util.Map;
4 import java.util.concurrent.ConcurrentHashMap;
5
6 import org.springframework.stereotype.Repository;
7
8 import com.example.lab10.model.Product;
9
10 import reactor.core.publisher.Flux;
11 import reactor.core.publisher.Mono;
12
13 /**
14  * ProductRepository - In-memory Reactive Repository
15  */
16 @Repository
17 public class ProductRepository {
18
19     // — In-memory storage —
20     private final Map<String, Product> store = new ConcurrentHashMap<>();
21
22     // — Constructor: ใส่ข้อมูลตัวอย่าง —
23     public ProductRepository() {
24         store.put(key: "1", new Product(id: "1", name: "iPhone 15 - Rattaphum 673380057-5 SEC 1",
25             category: "Electronics", brand: "Apple", stock: 50, price: 39900.0, discountType: "MEMBER"));
26         store.put(key: "2", new Product(id: "2", name: "MacBook Air M3",
27             category: "Electronics", brand: "Apple", stock: 20, price: 49900.0, discountType: "NONE"));
28         store.put(key: "3", new Product(id: "3", name: "Samsung Galaxy S24",
29             category: "Electronics", brand: "Samsung", stock: 30, price: 29900.0, discountType: "SEASONAL"));
30     }
31
32     // — 1. หา Product 1 รายการ —
33     public Mono<Product> findById(String id) {
34         Product product = store.get(id);
35         return product != null ? Mono.just(product) : Mono.empty();
36     }
37
38     // — 2. หา Product ทั้งหมด —
39     public Flux<Product> findAll() {
40         return Flux.fromIterable(store.values());
41     }
42
43     // — 3. บันทึก Product —
44     public Mono<Product> save(Product product) {
45         store.put(product.getId(), product);
46         return Mono.just(product);
47     }
48
49     // — 4. ลบ Product —
50     public Mono<Void> deleteById(String id) {
51         store.remove(id);
52         return Mono.empty();
53     }
54
55     // — 5. กรองตาม category —
56     public Flux<Product> findByCategory(String category) {
57         return findAll()
58             .filter(p -> p.getCategory().equalsIgnoreCase(category));
59     }
60 }
```

```
1  package com.example.lab10.service;
2
3  import com.example.lab10.model.Product;
4  import com.example.lab10.repository.ProductRepository;
5  import org.springframework.stereotype.Service;
6  import reactor.core.publisher.Flux;
7  import reactor.core.publisher.Mono;
8
9  import java.util.UUID;
10
11  /**
12   * ProductService - Business Logic Layer
13   */
14  @Service
15  public class ProductService {
16
17      // — Constructor Injection (DIP - SOLID) —————
18      private final ProductRepository repository;
19
20      public ProductService(ProductRepository repository) {
21          this.repository = repository;
22      }
23
24      // — 1. ดึง Product 1 รายการ —————
25      public Mono<Product> getById(String id) {
26          return repository.findById(id)
27              .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
28      }
29
30      // — 2. ดึง Product ทั้งหมด —————
31      public Flux<Product> getAll() {
32          return repository.findAll();
33      }
34
35      // — 3. บันทึก Product —————
36      public Mono<Product> save(Product product) {
37          if (product.getId() == null || product.getId().trim().isEmpty()) {
38              product.setId(UUID.randomUUID().toString());
39          }
40          return repository.save(product);
41      }
42
43      // — 4. ลบ Product —————
44      public Mono<Void> delete(String id) {
45          return repository.deleteById(id);
46      }
47
48      // — 5. กรองตาม category —————
49      public Flux<Product> getByCategory(String category) {
50          return repository.findByCategory(category);
51      }
52
53      // — 6. คำนวณราคาส่วนลด —————
54      public Mono<Double> getDiscountedPrice(String id) {
55          return getById(id)
56              .map(Product::getDiscountedPrice);
57      }
58  }
```

```
1  package com.example.lab10.controller;
2
3  import com.example.lab10.model.Product;
4  import com.example.lab10.service.ProductService;
5  import org.springframework.web.bind.annotation.*;
6  import reactor.core.publisher.Flux;
7  import reactor.core.publisher.Mono;
8
9  /**
10   * ProductController – Reactive REST Controller
11   */
12  @RestController
13  @RequestMapping("/products")
14  public class ProductController {
15
16      // — Constructor Injection (DIP – SOLID) —————
17      private final ProductService service;
18
19      public ProductController(ProductService service) {
20          this.service = service;
21      }
22
23      // =====
24      // ✅ ตัวอย่างที่ทำได้แล้ว
25      // =====
26
27      /**
28       * GET /products/{id}
29       */
30      @GetMapping("/{id}")
31      public Mono<Product> getById(@PathVariable String id) {
32          return service.getById(id);
33      }
34
35      // =====
36      // ❌ TODO: เติม method body ด้านล่างนี้ (เสร็จสมบูรณ์)
37      // =====
38
39      /**
40       * GET /products
41       */
42      @GetMapping
43      public Flux<Product> getAll() {
44          return service.getAll();
45      }
46
47      /**
48       * POST /products
```

```

49     */
50     @PostMapping
51     public Mono<Product> save(@RequestBody Product product) {
52         return service.save(product);
53     }
54
55     /**
56     * DELETE /products/{id}
57     */
58     @DeleteMapping("/{id}")
59     public Mono<Void> delete(@PathVariable String id) {
60         return service.delete(id);
61     }
62
63     /**
64     * GET /products/category/{category}
65     */
66     @GetMapping("/category/{category}")
67     public Flux<Product> getByCategory(@PathVariable String category) {
68         return service.getByCategory(category);
69     }
70
71     /**
72     * GET /products/{id}/price
73     */
74     @GetMapping("/{id}/price")
75     public Mono<Double> getDiscountedPrice(@PathVariable String id) {
76         return service.getDiscountedPrice(id);
77     }
78 }

```